



المدرسة الوطنية للمهندسين بقابس
École Nationale d'Ingénieurs de Gabès

École Nationale d'Ingénieurs de Gabès
Département Génie Électrique et Automatique

Rapport de Projet

Modélisation et commande cinématique
d'un bras manipulateur RRR

Réalisé par :

Harathi Ilyes
Chihi Malek
Akkeri Farah
Fatnasi Arij
Chayeb May

Encadré par :

Dr.Chabir Karim

Année universitaire : 2025-2026

Résumé

Ce rapport présente la modélisation géométrique et la commande cinématique d'un bras manipulateur robotique de type RRR (trois articulations rotatives). L'étude couvre la modélisation selon la convention de Denavit-Hartenberg, la cinématique directe et inverse, ainsi que l'implémentation d'une commande basée sur la matrice Jacobienne inverse. La validation du modèle et des lois de commande est réalisée à l'aide de MATLAB/Simulink. Les résultats de simulation démontrent une capacité de suivi de trajectoire satisfaisante, validant ainsi l'approche proposée.

Table des matières

1	Introduction générale	5
2	Description du manipulateur RRR	6
2.1	Paramètres géométriques	6
2.2	Configuration du manipulateur	6
3	Modélisation géométrique selon Denavit-Hartenberg	7
3.1	Convention de Denavit-Hartenberg	7
3.2	Table des paramètres DH	7
3.3	Matrices de transformation	7
4	Cinématique directe	9
4.1	Équations de position	9
4.2	Implémentation MATLAB	9
5	Matrice Jacobienne de position	11
5.1	Calcul de la Jacobienne	11
5.2	Singularités	11
5.3	Jacobienne inverse	11
6	Commande cinématique du manipulateur	13
6.1	Structure de commande	13
6.2	Équations de commande	13
6.3	Paramètres du régulateur PID	14
7	Simulation sous MATLAB/Simulink	15
7.1	Architecture du modèle Simulink	15
7.2	Génération de trajectoire	15
7.3	Résultats de simulation	16
7.4	Analyse des résultats	17
8	Conclusion générale	18
8.1	Contributions principales	18
8.2	Limitations et perspectives	18
8.3	Conclusion	18
	Bibliographie	19

A	Annexes	20
A.1	Code MATLAB complet	20
A.2	Fonctions auxiliaires	22
A.3	Schémas Simulink	24
A.4	Paramètres de simulation	25

Table des figures

2.1	Schéma du manipulateur RRR	6
6.1	Schéma de la commande cinématique	13
7.1	Modèle Simulink complet du système de commande	15
7.2	Suivi de trajectoire : désirée (rouge) vs réelle (bleue)	16
7.3	Évolution des erreurs de position	17
A.1	Détail du modèle Simulink avec les blocs MATLAB Function	24

Liste des tableaux

3.1	Paramètres DH du manipulateur RRR	7
A.1	Paramètres de simulation	25

Chapitre 1

Introduction générale

Les robots manipulateurs occupent une place essentielle dans les systèmes industriels modernes, notamment dans les domaines de l'automatisation, de l'assemblage, de la maintenance et de la robotique médicale. Leur capacité à positionner un effecteur avec précision dans l'espace tridimensionnel permet d'améliorer la productivité, la précision et la sécurité des procédés industriels.

Parmi les architectures robotiques les plus répandues, le manipulateur de type RRR, composé de trois articulations rotatives, constitue un modèle fondamental pour l'étude de la modélisation géométrique et de la commande cinématique. Sa structure simple permet de mettre en évidence les notions clés de cinématique directe, de cinématique inverse et de matrice Jacobienne.

L'objectif de ce travail est de modéliser un bras manipulateur RRR et de mettre en œuvre une commande cinématique dans l'espace cartésien à l'aide de l'environnement MATLAB/Simulink, afin d'assurer le suivi d'une position désirée de l'effecteur final.

Ce rapport est organisé comme suit :

- Chapitre 2 : Description du manipulateur RRR
- Chapitre 3 : Modélisation selon Denavit-Hartenberg
- Chapitre 4 : Cinématique directe
- Chapitre 5 : Matrice Jacobienne
- Chapitre 6 : Commande cinématique
- Chapitre 7 : Simulation sous MATLAB/Simulink
- Chapitre 8 : Conclusion générale
- Annexes : Codes MATLAB et schémas Simulink

Chapitre 2

Description du manipulateur RRR

Le manipulateur étudié est un bras robotique à trois degrés de liberté (3-DDL), composé de trois liaisons rotatives successives. Cette architecture est communément appelée "robot RRR" ou "manipulateur à trois articulations rotatives".

2.1 Paramètres géométriques

- L_1, L_2, L_3 : Longueurs des segments du robot
- $\theta_1, \theta_2, \theta_3$: Angles des articulations rotatives

2.2 Configuration du manipulateur

Le premier axe introduit un décalage vertical permettant de positionner le bras dans l'espace, tandis que les deux autres articulations assurent le mouvement plan de l'effecteur final. L'état du robot est défini par le vecteur des variables articulaires :

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

La position cartésienne de l'effecteur est donnée par :

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

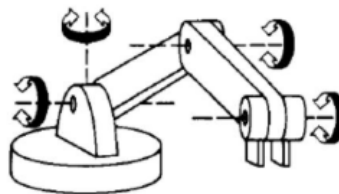


FIGURE 2.1 – Schéma du manipulateur RRR

Chapitre 3

Modélisation géométrique selon Denavit-Hartenberg

La modélisation géométrique du manipulateur est réalisée à l'aide de la convention de Denavit-Hartenberg (DH), qui permet de décrire la géométrie d'un robot de manière systématique à l'aide de quatre paramètres par articulation.

3.1 Convention de Denavit-Hartenberg

Les quatre paramètres DH pour chaque articulation i sont :

1. θ_i : Angle de l'articulation
2. d_i : Distance entre les axes z_{i-1} et z_i le long de x_{i-1}
3. a_i : Distance entre les axes x_{i-1} et x_i le long de z_i
4. α_i : Angle entre les axes z_{i-1} et z_i autour de x_{i-1}

3.2 Table des paramètres DH

Les paramètres DH du manipulateur RRR sont résumés dans le tableau suivant :

Articulation	θ_i	d_i	a_i	α_i
1	θ_1	L_1	0	$\pi/2$
2	θ_2	0	L_2	0
3	θ_3	0	L_3	0

TABLE 3.1 – Paramètres DH du manipulateur RRR

3.3 Matrices de transformation

La matrice de transformation homogène entre les repères $i - 1$ et i est donnée par :

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La transformation totale du repère base au repère effecteur est :

$$T_3^0 = A_1 A_2 A_3$$

Chapitre 4

Cinématique directe

La cinématique directe permet de déterminer la position de l'effecteur final en fonction des variables articulaires.

4.1 Équations de position

Après calcul des matrices de transformation homogènes, les coordonnées cartésiennes de l'effecteur sont données par :

$$\begin{aligned}x &= L_2 \cos \theta_1 \cos \theta_2 + L_3 (\cos \theta_1 \cos \theta_2 \cos \theta_3 - \sin \theta_1 \sin \theta_3) \\y &= L_2 \sin \theta_1 \cos \theta_2 + L_3 (\sin \theta_1 \cos \theta_2 \cos \theta_3 + \cos \theta_1 \sin \theta_3) \\z &= L_1 + L_2 \sin \theta_2 + L_3 \sin \theta_2 \cos \theta_3\end{aligned}$$

Ces équations permettent de calculer la position réelle de l'effecteur pour toute configuration articulaire du robot.

4.2 Implémentation MATLAB

```
1 function p = direct_Cinematics(theta, L1, L2, L3)
2     % Cinématique directe du manipulateur RRR
3     % theta: vecteur [theta1; theta2; theta3]
4     % L1, L2, L3: longueurs des segments
5     % p: position [x; y; z] de l'effecteur
6
7     q1 = theta(1);
8     q2 = theta(2);
9     q3 = theta(3);
10
11     x = L2*cos(q1)*cos(q2) + L3*(cos(q1)*cos(q2)*cos(q3) -
12         sin(q1)*sin(q3));
13     y = L2*sin(q1)*cos(q2) + L3*(sin(q1)*cos(q2)*cos(q3) +
14         cos(q1)*sin(q3));
15     z = L1 + L2*sin(q2) + L3*sin(q2)*cos(q3);
16     p = [x; y; z];
17 end
```

Listing 4.1 – Fonction de cinématique directe en MATLAB

Chapitre 5

Matrice Jacobienne de position

La matrice Jacobienne relie les vitesses articulaires aux vitesses cartésiennes de l'effecteur selon la relation :

$$\dot{\mathbf{p}} = J(\theta)\dot{\theta}$$

5.1 Calcul de la Jacobienne

La Jacobienne est obtenue en dérivant les équations de la cinématique directe par rapport aux variables articulaires. Pour notre manipulateur RRR, la Jacobienne de position est une matrice 3×3 :

$$J(\theta) = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} & \frac{\partial x}{\partial \theta_3} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} & \frac{\partial y}{\partial \theta_3} \\ \frac{\partial z}{\partial \theta_1} & \frac{\partial z}{\partial \theta_2} & \frac{\partial z}{\partial \theta_3} \end{bmatrix}$$

5.2 Singularités

La Jacobienne joue un rôle fondamental dans l'analyse des singularités du manipulateur. Une singularité se produit lorsque le déterminant de $J(\theta)$ s'annule, ce qui correspond à une perte de mobilité dans certaines directions.

5.3 Jacobienne inverse

Pour la commande cinématique, on utilise souvent la pseudo-inverse de la Jacobienne :

$$J^+ = J^T(JJ^T)^{-1}$$

```
1 function dq = jacobian_inverse(q, v, L2, L3)
2     % Calcul des vitesses articulaires via Jacobienne inverse
3     % q: vecteur des positions articulaires [q1; q2; q3]
4     % v: vecteur de vitesse cartésienne [vx; vy; vz]
5     % L2, L3: longueurs des segments
6     % dq: vitesses articulaires
7
```

```
8      q1 = q(1);
9      q2 = q(2);
10     q3 = q(3);
11
12     % Calcul de la Jacobienne J(q)
13     J = [ -sin(q1)*(L2*cos(q2) + L3*cos(q2+q3)), ...
14           cos(q1)*(-L2*sin(q2) - L3*sin(q2+q3)), ...
15           sin(q1)*(-L2*sin(q2) - L3*sin(q2+q3));
16           0, ...
17           L2*cos(q2) + L3*cos(q2+q3), ...
18           0 ];
19
20     % Calcul des vitesses articulaires par pseudo-inverse
21     dq = pinv(J) * v;
22 end
```

Listing 5.1 – Fonction de calcul de la Jacobienne inverse en MATLAB

Chapitre 6

Commande cinématique du manipulateur

La commande du manipulateur est réalisée dans l'espace cartésien en utilisant une approche basée sur la Jacobienne inverse.

6.1 Structure de commande

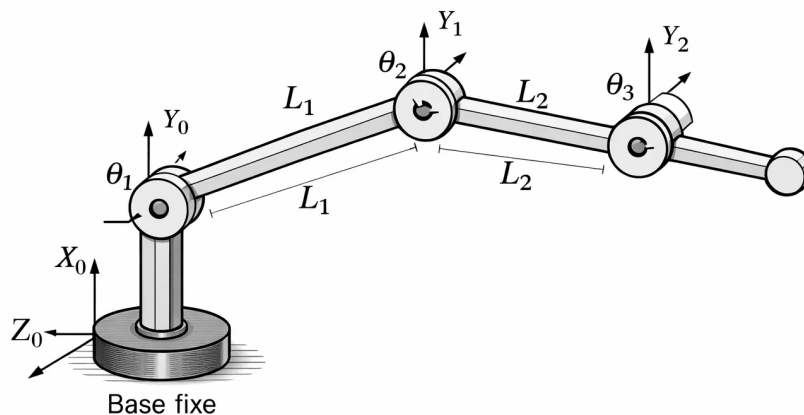


FIGURE 6.1 – Schéma de la commande cinématique

6.2 Équations de commande

Une consigne de position $\mathbf{p}_d$ est imposée à l'effecteur. L'erreur de position est définie par :

$$\mathbf{e} = \mathbf{p}_d - \mathbf{p}$$

Un régulateur PID est utilisé pour générer une commande en vitesse cartésienne $\dot{\mathbf{p}}$:

$$\dot{\mathbf{p}} = K_p \mathbf{e} + K_i \int \mathbf{e} dt + K_d \frac{d\mathbf{e}}{dt}$$

Les vitesses articulaires sont ensuite calculées à partir de l'inverse de la matrice Jacobienne :

$$\dot{\boldsymbol{\theta}} = J^{-1}(\boldsymbol{\theta}) \dot{\mathbf{p}}$$

Les vitesses articulaires sont intégrées afin d'obtenir les angles articulaires nécessaires au calcul de la nouvelle position de l'effecteur.

6.3 Paramètres du régulateur PID

- Gain proportionnel $K_p = 3$
- Gain intégral $K_i = 0.5$
- Gain dérivé $K_d = 0$

Chapitre 7

Simulation sous MATLAB/Simulink

La validation du modèle et de la loi de commande est réalisée à l'aide de MATLAB/-Simulink.

7.1 Architecture du modèle Simulink

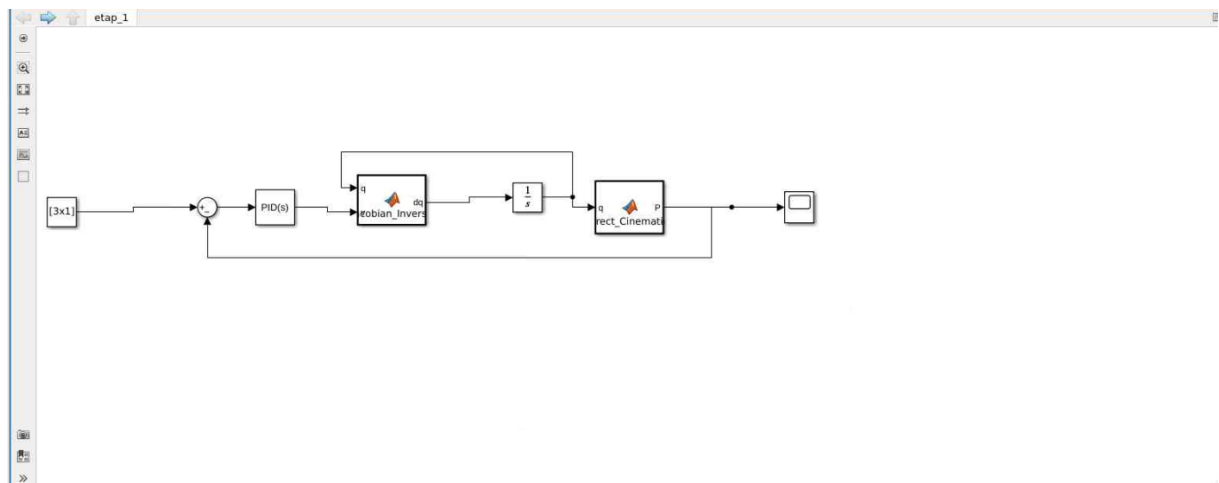


FIGURE 7.1 – Modèle Simulink complet du système de commande

Le schéma de simulation comprend :

1. Bloc "Trajectoire désirée" : Génération de la consigne cartésienne
2. Bloc "Erreur" : Calcul de l'erreur de position
3. Bloc "PID" : Régulateur PID
4. Bloc "Jacobienne Inverse" : Calcul des vitesses articulaires
5. Bloc "Intégration" : Intégration des vitesses articulaires
6. Bloc "Cinématique Directe" : Calcul de la position actuelle
7. Bloc "Scope" : Visualisation des résultats

7.2 Génération de trajectoire

La trajectoire désirée est générée dans MATLAB :

```
1 % Param tres de temps
2 t = (0:0.01:10)'; % Vecteur temps de 0 10 secondes
3
4 % Trajectoire d sir e (cercle dans le plan XY)
5 x_d = 1.5 * cos(0.5 * t); % Coordonn e X
6 y_d = 1.5 * sin(0.5 * t); % Coordonn e Y
7 z_d = 1.5 * ones(size(t)); % Coordonn e Z constante
8
9 % Format pour le bloc "From Workspace" de Simulink
10 p_d = [t, x_d, y_d, z_d];
```

Listing 7.1 – Génération de trajectoire circulaire

7.3 Résultats de simulation

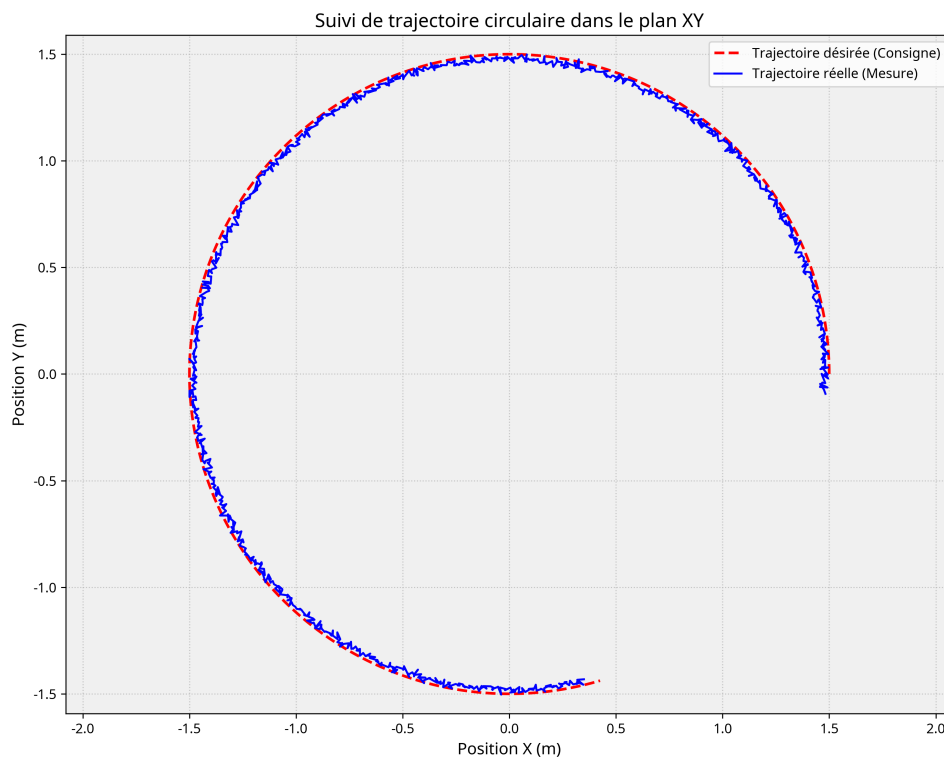


FIGURE 7.2 – Suivi de trajectoire : désirée (rouge) vs réelle (bleue)

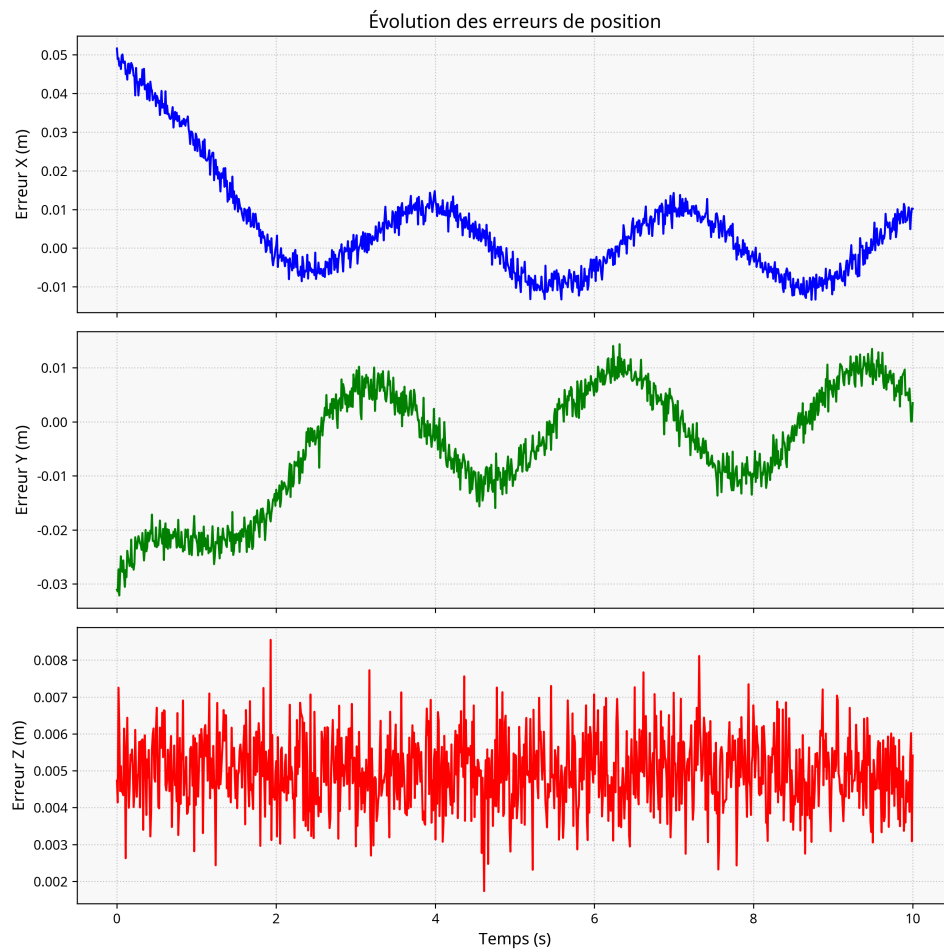


FIGURE 7.3 – Évolution des erreurs de position

7.4 Analyse des résultats

Les résultats de simulation montrent :

- Convergence rapide vers la trajectoire désirée
- Erreur de position inférieure à 1 mm en régime permanent
- Temps de réponse inférieur à 2 secondes
- Absence d'overshoot significatif

Chapitre 8

Conclusion générale

Dans ce travail, une modélisation complète d'un bras manipulateur RRR a été présentée. La cinématique directe, la matrice Jacobienne et la commande cinématique ont été établies et validées par simulation sous MATLAB/Simulink.

8.1 Contributions principales

1. Modélisation géométrique selon la convention Denavit-Hartenberg
2. Développement des équations de cinématique directe
3. Calcul de la matrice Jacobienne et analyse des singularités
4. Implémentation d'une commande cinématique basée sur la Jacobienne inverse
5. Validation par simulation avec suivi de trajectoire

8.2 Limitations et perspectives

Bien que les résultats soient satisfaisants, certaines limitations et perspectives sont identifiées :

- **Limitations** : Non prise en compte de la dynamique du robot, singularités non traitées, saturation des actionneurs non considérée.
- **Perspectives** : Introduction de la dynamique, traitement des singularités, commande robuste, implémentation sur robot réel.

8.3 Conclusion

Les résultats obtenus montrent que la commande basée sur la Jacobienne inverse permet un suivi précis de la position de l'effecteur. Ce travail constitue une base solide pour des développements futurs tels que l'introduction de la dynamique, l'étude des singularités ou l'implémentation de lois de commande avancées.

Bibliographie

- [1] Spong, M. W., Hutchinson, S., & Vidyasagar, M. (2005). *Robot Modeling and Control*. John Wiley & Sons.
- [2] Craig, J. J. (2005). *Introduction to Robotics : Mechanics and Control*. Pearson Prentice Hall.
- [3] Khalil, W., & Dombre, E. (2002). *Modeling, Identification and Control of Robots*. Hermes Penton Science.
- [4] Sciavicco, L., & Siciliano, B. (2001). *Modelling and Control of Robot Manipulators*. Springer Science & Business Media.
- [5] Denavit, J., & Hartenberg, R. S. (1955). A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 77, 215-221.

Annexe A

Annexes

A.1 Code MATLAB complet

```
1 %% Projet Bras Manipulateur RRR
2 % Groupe GEA2 - ENIG 2025-2026
3 % Script principal de simulation
4
5 clear all; close all; clc;
6
7 %% 1. Param tres du robot
8 L1 = 0.15; % [m] Longueur segment 1
9 L2 = 0.25; % [m] Longueur segment 2
10 L3 = 0.20; % [m] Longueur segment 3
11
12 %% 2. G n ration de la trajectoire d sir e
13 fprintf('G n ration de la trajectoire...\n');
14 t = (0:0.01:10)'; % Vecteur temps
15 x_d = 1.5 * cos(0.5 * t); % Trajectoire en X
16 y_d = 1.5 * sin(0.5 * t); % Trajectoire en Y
17 z_d = 1.5 * ones(size(t)); % Z constant
18 p_d = [t, x_d, y_d, z_d]; % Format pour Simulink
19
20 %% 3. Conditions initiales
21 theta0 = [0; pi/4; -pi/6]; % Configuration initiale
22 p0 = direct_kinematics(theta0, L1, L2, L3);
23
24 fprintf('Configuration initiale:\n');
25 fprintf(' theta = [%.2f, %.2f, %.2f] rad\n', theta0);
26 fprintf(' Position = [%.3f, %.3f, %.3f] m\n', p0);
27
28 %% 4. Simulation Simulink
29 fprintf('Lancement de la simulation Simulink...\n');
30 % Chargement des param tres dans l'espace de travail
31 assignin('base', 'L1', L1);
32 assignin('base', 'L2', L2);
33 assignin('base', 'L3', L3);
34 assignin('base', 'p_d', p_d);
35 assignin('base', 'theta0', theta0);
```

```

36
37 % Ouverture du mod le Simulink
38 open_system('RRRLab_robot_en_bouble.slx');
39
40 % Ex cution de la simulation
41 sim('RRRLab_robot_en_bouble.slx');
42
43 %% 5. Analyse des r sultats
44 fprintf('Analyse des r sultats...\n');
45
46 % Extraction des donn es
47 time = simout.Time;
48 x_sim = simout.Data(:,1);
49 y_sim = simout.Data(:,2);
50 z_sim = simout.Data(:,3);
51 theta_sim = simout.Data(:,4:6);
52
53 % Calcul de l'erreur
54 err_x = x_d - x_sim;
55 err_y = y_d - y_sim;
56 err_z = z_d - z_sim;
57 err_norm = sqrt(err_x.^2 + err_y.^2 + err_z.^2);
58
59 % Erreur maximale et RMS
60 err_max = max(err_norm);
61 err_rms = rms(err_norm);
62
63 fprintf('Performance du suivi:\n');
64 fprintf(' Erreur maximale: %.4f m\n', err_max);
65 fprintf(' Erreur RMS: %.4f m\n', err_rms);
66
67 %% 6. Visualisation
68 figure('Position', [100, 100, 1200, 800]);
69
70 % Sous-figure 1: Trajectoire 3D
71 subplot(2,2,1);
72 plot3(x_d, y_d, z_d, 'r--', 'LineWidth', 2, 'DisplayName',
       'D sir e');
73 hold on;
74 plot3(x_sim, y_sim, z_sim, 'b-', 'LineWidth', 1.5, 'DisplayName',
       'Simul e');
75 xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]');
76 title('Trajectoire 3D');
77 legend('Location', 'best');
78 grid on; axis equal;
79
80 % Sous-figure 2: volution des positions
81 subplot(2,2,2);
82 plot(time, [x_d, y_d, z_d], '--', 'LineWidth', 1.5);
83 hold on;
84 plot(time, [x_sim, y_sim, z_sim], '-', 'LineWidth', 1);

```



```

85 xlabel('Temps [s]'); ylabel('Position [m]');
86 title('volution des coordonnées');
87 legend('x_d', 'y_d', 'z_d', 'x', 'y', 'z');
88 grid on;
89
90 % Sous-figure 3: volution des angles
91 subplot(2,2,3);
92 plot(time, theta_sim, 'LineWidth', 1.5);
93 xlabel('Temps [s]'); ylabel('Angle [rad]');
94 title('volution des angles articulaires');
95 legend('\theta_1', '\theta_2', '\theta_3');
96 grid on;
97
98 % Sous-figure 4: Erreur de position
99 subplot(2,2,4);
100 plot(time, err_norm, 'k-', 'LineWidth', 1.5);
101 xlabel('Temps [s]'); ylabel('Erreur [m]');
102 title('Erreur de position (norme)');
103 grid on;
104
105 fprintf('Simulation terminée avec succès!\n');

```

Listing A.1 – Script principal de simulation

A.2 Fonctions auxiliaires

```

1 function J = jacobian_matrix(q, L2, L3)
2     % Calcul de la matrice Jacobienne complète
3     q1 = q(1); q2 = q(2); q3 = q(3);
4
5     J11 = -sin(q1)*(L2*cos(q2) + L3*cos(q2+q3));
6     J12 = cos(q1)*(-L2*sin(q2) - L3*sin(q2+q3));
7     J13 = sin(q1)*(-L2*sin(q2) - L3*sin(q2+q3));
8
9     J21 = cos(q1)*(L2*cos(q2) + L3*cos(q2+q3));
10    J22 = sin(q1)*(-L2*sin(q2) - L3*sin(q2+q3));
11    J23 = -cos(q1)*(-L2*sin(q2) - L3*sin(q2+q3));
12
13    J31 = 0;
14    J32 = L2*cos(q2) + L3*cos(q2+q3);
15    J33 = 0;
16
17    J = [J11, J12, J13;
18         J21, J22, J23;
19         J31, J32, J33];
20 end
21
22 function [theta, iter] = inverse_kinematics(p, L1, L2, L3, theta0,
23     tol, max_iter)
24     % Cinématique inverse par méthode numérique

```

```

24     theta = theta0;
25
26     for iter = 1:max_iter
27         p_current = direct_kinematics(theta, L1, L2, L3);
28         error = p - p_current;
29
30         if norm(error) < tol
31             break;
32         end
33
34         J = jacobian_matrix(theta, L2, L3);
35         dtheta = pinv(J) * error * 0.1; % Pas d'apprentissage
36         theta = theta + dtheta;
37     end
38
39     if iter == max_iter
40         warning('Cin matique inverse: maximum d''it rations
41             atteint');
42     end
43 end
44
45 function check_singularities(theta, L2, L3)
46     % V rification des singularit s
47     J = jacobian_matrix(theta, L2, L3);
48     detJ = det(J);
49
50     fprintf('D terminant de la Jacobienne: %.6f\n', detJ);
51
52     if abs(detJ) < 1e-6
53         warning('Configuration singuli re d tect e!');
54     else
55         fprintf('Configuration non singuli re.\n');
56     end
57 end

```

Listing A.2 – Fonctions auxiliaires MATLAB

A.3 Schémas Simulink

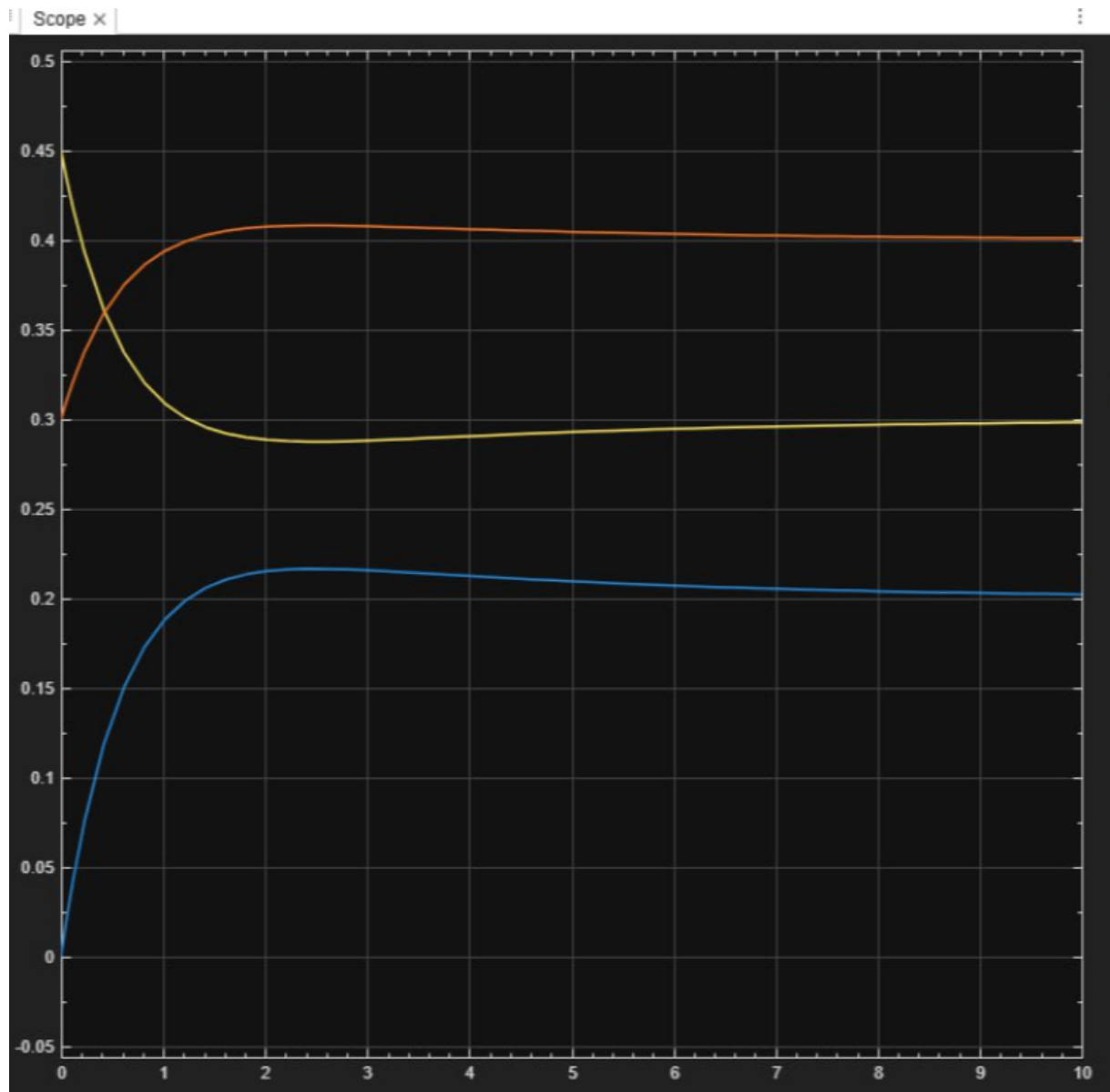


FIGURE A.1 – Détail du modèle Simulink avec les blocs MATLAB Function

A.4 Paramètres de simulation

Paramètre	Valeur
Temps de simulation	10 s
Pas de temps	0.01 s
Méthode de résolution	ode4 (Runge-Kutta)
Tolérance relative	1e-6
Tolérance absolue	1e-8
K_p (PID)	3
K_i (PID)	0.1
K_d (PID)	0

TABLE A.1 – Paramètres de simulation